

JAVASCRIPT

Interview Questions & Answers

100 concise, interview-ready answers for JavaScript developers

Interview style: definition first, key points next, and short examples where they add value. Built for quick revision and confident verbal answers.

Prepared as a practical revision guide

Quick Index

- **01-10 Fundamentals & Core Language Concepts**
- **11-20 Functions, Events & Asynchronous Basics**
- **21-30 Memory, Event Loop, Arrays & Variables**
- **31-40 DOM, Storage, Functions & Scope**
- **41-50 Strings, Objects, Call Context & Event Loop**
- **51-60 Modern Syntax, JSON & Common Array Techniques**
- **61-70 Modules, Web APIs, Objects & Arrays**
- **71-80 Browser APIs, Collections & Object Utilities**
- **81-90 Performance, Inheritance & Advanced Functions**
- **91-100 Production Concepts & Promise Patterns**

Note: Your supplied list contained 95 questions because numbers 28, 37, 44, 69 and 77 were missing. Five closely related questions were added so the guide contains a complete 100-question set.

01. Fundamentals & Core Language Concepts

Questions 1-10

Q1. What is JavaScript?

JavaScript is a high-level, dynamically typed programming language used to add interactivity and dynamic behavior to web applications. It runs in browsers and can also be used on the server side with environments such as Node.js.

Example

```
const message = "Hello, JavaScript!";
```

Q2. What are the data types in JavaScript?

A data type defines the kind of value a variable can hold and how that value can be used. JavaScript has 8 data types, divided into primitive and non-primitive types.

Primitive types represent single values and are immutable: String, Number, BigInt, Boolean, Undefined, Null, and Symbol. The non-primitive type is Object, which includes structures such as objects, arrays, and functions.

Example

```
let name = "John"; // String
let age = 25;      // Number
let active = true; // Boolean
let user = {};     // Object
```

Q3. What is the difference between null and undefined?

undefined means a value has not been assigned, while null is an intentional assignment that represents the absence of a value.

Example: let x; gives undefined, while let y = null; explicitly sets the value to null.

Q4. Explain the concept of hoisting in JavaScript.

Hoisting is JavaScript's behavior of processing declarations before executing the code. Function declarations can be called before they appear in the code. var is hoisted and initialized with undefined, while let and const are hoisted but remain unavailable in the Temporal Dead Zone until their declaration is reached.

Example

```
sayHello();
function sayHello() { console.log("Hello"); }
```

Q5. What is a closure in JavaScript?

A closure is created when an inner function remembers and can access variables from the outer function where it was created, even after the outer function has finished executing. In simple terms, a closure lets a function remember data from its surrounding scope.

Example

```
function counter() {  
  let count = 0;  
  return () => ++count;  
}  
const next = counter();  
next(); // 1  
next(); // 2
```

Q6. What is the difference between “==” and “===” operators in JavaScript?

== is loose equality, so JavaScript may perform type conversion before comparing values. === is strict equality, so it compares both value and type without implicit type conversion. In modern JavaScript, === is generally preferred because its behavior is more predictable.

Example

```
5 == "5" // true  
5 === "5" // false
```

Q7. Explain the concept of prototypal inheritance in JavaScript.

Prototypal inheritance is JavaScript's mechanism for sharing properties and methods between objects. When a property is not found on an object, JavaScript looks for it in the object's prototype and continues through the prototype chain.

Example

```
const parent = { greet() { console.log("Hi"); } };  
const child = Object.create(parent);  
child.greet(); // Hi
```

Q8. What are the different ways to define a function in JavaScript?

Common ways include function declarations, function expressions, arrow functions, and object or class methods. They differ in areas such as hoisting, this behavior, and constructor usage.

Example

```
function add(a, b) { return a + b; }  
const multiply = (a, b) => a * b;
```

Q9. How does event delegation work in JavaScript?

Event delegation attaches one event listener to a parent element instead of separate listeners to each child. It relies on event bubbling to determine which child triggered the event. It reduces the number of listeners and works well for dynamically added elements.

Example

```
list.addEventListener("click", e => {
  if (e.target.matches("li")) console.log(e.target.textContent);
});
```

Q10. What is the purpose of the “this” keyword in JavaScript?

this refers to the object or context associated with the current function call. Its value depends mainly on how the function is called. Regular functions can receive different this values, while arrow functions do not have their own this and inherit it from the surrounding scope.

Example

```
const user = {
  name: "John",
  greet() { console.log(this.name); }
};
user.greet(); // John
```

02. Functions, Events & Asynchronous Basics

Questions 11-20

Q11. What are the different ways to create objects in JavaScript?

Objects can be created using object literals, the new Object() constructor, constructor functions with new, classes with new, Object.create(), and factory functions. Object literals and classes are common in modern code, while Object.create() is useful when you want to control the prototype directly.

Example

```
const a = { name: "John" };
const b = Object.create(a);
```

Q12. Explain the concept of callback functions in JavaScript.

A callback is a function passed to another function so it can be executed later or when a particular operation is complete. Callbacks are common in array methods, event handling, and asynchronous code.

Example

```
setTimeout(() => {
  console.log("Done");
}, 1000);
```

Q13. What is event bubbling and event capturing in JavaScript?

Event bubbling moves an event from the target element upward through its ancestors. Event capturing moves from the outer ancestor down toward the target. By default, most event listeners use bubbling. The capture phase can be enabled with `{ capture: true }`.

Example

```
parent.addEventListener("click", handler, { capture: true });
```

Q14. What is the purpose of the “bind” method in JavaScript?

`bind()` creates a new function with this permanently set to a specified value and can also preset arguments. It is useful when a function needs a fixed context, especially when passing methods as callbacks.

Example

```
const greet = user.greet.bind(user);  
greet();
```

Q15. Explain the concept of AJAX in JavaScript.

AJAX stands for Asynchronous JavaScript and XML. It is a technique for communicating with a server asynchronously so a web page can fetch or send data without a full page reload. Today, AJAX commonly uses `fetch()` and JSON rather than XML.

Example

```
fetch("/api/users")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Q16. What is the “typeof” operator used for?

`typeof` returns a string describing the type of a value. It is commonly used to check primitive types and whether a value is a function. One well-known quirk is that `typeof null` returns `"object"`.

Example

```
typeof 42;    // "number"  
typeof "Hi";  // "string"  
typeof null;  // "object"
```

Q17. How does JavaScript handle errors and exceptions?

JavaScript uses exceptions to represent runtime errors. You can handle them with `try...catch`, optionally clean up with `finally`, and create custom errors with `throw`. Promise-based code can use `catch()` or `try...catch` with `async/await`.

Example

```
try {  
  riskyOperation();  
} catch (error) {  
  console.error(error);  
}
```

Q18. Explain the concept of event-driven programming in JavaScript.

Event-driven programming is a model where program flow is driven by events such as clicks, keyboard input, network responses, or timers. JavaScript registers handlers for these events and runs the appropriate callback when an event occurs.

Example

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

Q19. What is the purpose of the “async” and “await” keywords in JavaScript?

async and await provide a cleaner way to work with Promises. An async function always returns a Promise, and await pauses execution inside that async function until the Promise settles, making asynchronous code easier to read and maintain.

Example

```
async function load() {  
  const response = await fetch("/api/users");  
  return response.json();  
}
```

Q20. What is the difference between a deep copy and a shallow copy in JavaScript?

A shallow copy copies the top-level properties but keeps references to nested objects. A deep copy recursively copies nested data so the copied structure does not share those nested references with the original.

Example

```
const shallow = { ...user };  
const deep = structuredClone(user);
```

03. Memory, Event Loop, Arrays & Variables

Questions 21-30

Q21. How does JavaScript handle memory management?

JavaScript automatically manages memory through allocation and garbage collection. Objects and values use memory when created, and the garbage collector reclaims memory that is no longer reachable by the program. Developers mainly focus on avoiding unnecessary references and memory leaks.

Q22. Explain the concept of the event loop in JavaScript.

The event loop coordinates synchronous code and asynchronous callbacks in JavaScript. The call stack executes current code, while asynchronous work is handled outside the stack; when the stack is empty, queued callbacks are moved into it for execution. Promise reactions use the microtask queue, which is processed before the next macrotask such as a timer callback.

Example

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// A, D, C, B
```

Q23. What is the purpose of the “map” method in JavaScript?

map() creates a new array by applying a callback function to every element of the original array. It is mainly used when you want to transform each item without changing the original array.

Example

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
// [2, 4, 6]
```

Q24. What is a promise in JavaScript?

A Promise is an object that represents the eventual result of an asynchronous operation. It has three states: pending, fulfilled, and rejected. Promises can be handled with then(), catch(), and finally().

Example

```
const promise = fetch("/api/users");
promise.then(res => res.json()).catch(console.error);
```

Q25. How do you handle errors in promises?

Promise errors can be handled with catch(), by providing a rejection handler to then(), or with try...catch when using await. A finally() block can run cleanup code regardless of success or failure.

Example

```
fetch("/api/data")
  .then(res => res.json())
  .catch(err => console.error(err));
```

Q26. Explain the concept of currying in JavaScript.

Currying transforms a function that takes multiple arguments into a sequence of functions, each taking one argument. It is useful for partial application and creating reusable specialized functions.

Example

```
const add = a => b => a + b;
add(2)(3); // 5
```

Q27. What is the purpose of the “reduce” method in JavaScript?

reduce() processes an array and combines its elements into a single accumulated result. The result can be a number, string, object, array, or any other value, depending on the reducer logic.

Example

```
const total = [1, 2, 3].reduce((sum, n) => sum + n, 0);  
// 6
```

Q28. What is the difference between a function declaration and a function expression?

A function declaration defines a function directly with the function keyword and is fully hoisted. A function expression creates a function as part of an expression, usually assigned to a variable, and the variable's declaration rules determine when it can be accessed.

Example

```
hello();  
function hello() {}  
  
const hi = function() {};
```

Q29. What are the different types of loops in JavaScript?

Common loop forms are for, while, do...while, for...of, and for...in. for...of is generally used to iterate over iterable values such as arrays, while for...in is used to enumerate object property keys.

Example

```
for (const value of [1, 2, 3]) {  
  console.log(value);  
}
```

Q30. What is the difference between “let,” “const,” and “var” in JavaScript?

let and const are block-scoped, while var is function-scoped. let allows reassignment, const does not allow reassignment of the binding, and var can be redeclared in the same scope. In modern JavaScript, let and const are generally preferred over var.

Example

```
let count = 1;  
count = 2;  
const limit = 10;  
// limit = 20 -> error
```

04. DOM, Storage, Functions & Scope

Questions 31-40

Q31. Explain the concept of event propagation in JavaScript.

Event propagation is the process by which an event travels through the DOM. It has three phases: capturing, target, and bubbling. Understanding propagation helps with event delegation and controlling how parent and child elements respond to the same event.

Q32. What are the different ways to manipulate the DOM in JavaScript?

Common DOM manipulation methods include selecting elements with `querySelector/querySelectorAll`, changing text with `textContent`, changing attributes with `setAttribute`, modifying classes with `classList`, creating elements with `createElement`, and inserting or removing nodes with `append`, `prepend`, and `remove`.

Example

```
const el = document.querySelector("#title");
el.textContent = "Updated";
el.classList.add("active");
```

Q33. What is the purpose of the “localStorage” and “sessionStorage” objects?

`localStorage` stores key-value data that persists across browser sessions, while `sessionStorage` stores data for the lifetime of the current browser tab or session. Both store strings and are scoped to the same origin.

Example

```
localStorage.setItem("theme", "dark");
sessionStorage.setItem("step", "2");
```

Q34. How do you handle asynchronous operations in JavaScript?

Asynchronous operations can be handled using callbacks, Promises, and `async/await`. In modern JavaScript, Promises and `async/await` are generally preferred because they improve readability and make error handling easier.

Q35. What is the purpose of the “forEach” method in JavaScript?

`forEach()` executes a callback once for each element in an array. It is useful for performing side effects such as logging, updating the DOM, or calling another function. It does not create a new array and cannot be stopped with `break`.

Example

```
[1, 2, 3].forEach(n => console.log(n));
```

Q36. What are the differences between “let” and “var” in JavaScript?

let is block-scoped, while var is function-scoped. let cannot be redeclared in the same scope and is subject to the Temporal Dead Zone before declaration. var can be redeclared and is initialized as undefined when hoisted.

Example

```
if (true) {  
  let a = 1;  
  var b = 2;  
}  
// a is unavailable here; b is available here
```

Q37. What is optional chaining and nullish coalescing in JavaScript?

Optional chaining (?.) safely accesses nested properties or calls when an earlier value may be null or undefined. Nullish coalescing (??) provides a fallback only when the left side is null or undefined, not for other falsy values such as 0 or false.

Example

```
const city = user?.address?.city ?? "Unknown";
```

Q38. What is the purpose of the “splice” method in JavaScript arrays?

splice() changes the original array by adding, removing, or replacing elements at a specified index. Because it mutates the array, it should be used carefully when immutability is important.

Example

```
const a = [1, 2, 3, 4];  
a.splice(1, 2);  
// a is [1, 4]
```

Q39. What is a generator function in JavaScript?

A generator function is declared with function* and can pause and resume execution using yield. Calling it returns a generator object that produces values lazily through next(). Generators are useful for custom iteration and controlled sequences.

Example

```
function* ids() {  
  yield 1;  
  yield 2;  
}  
[...ids()]; // [1, 2]
```

Q40. How does JavaScript handle variable scoping?

JavaScript has global scope, function scope, block scope, and module scope. let and const are block-scoped, var is function-scoped, and lexical scope determines which variables a function can access based on where it is defined.

05. Strings, Objects, Call Context & Event Loop

Questions 41-50

Q41. What is the purpose of the “split” method in JavaScript?

split() divides a string into an array based on a specified separator. It is commonly used for parsing text or converting delimited strings into arrays.

Example

```
"a,b,c".split(",");  
// ["a", "b", "c"]
```

Q42. What is the difference between a deep clone and a shallow clone of an object?

A shallow clone copies only the top level and keeps references to nested objects. A deep clone creates independent nested copies. structuredClone() is a modern built-in option for many common cloneable values.

Example

```
const shallow = { ...obj };  
const deep = structuredClone(obj);
```

Q43. Explain the concept of the event delegation pattern.

The event delegation pattern centralizes event handling on a common ancestor instead of attaching listeners to every child. It uses event bubbling and event.target or closest() to identify the actual element that triggered the event.

Q44. What is the difference between object equality and value equality in JavaScript?

For objects and arrays, == and === compare references, not their contents. Two separate objects with the same data are still different references. Value-based comparison requires explicit logic or a suitable utility.

Example

```
{ } === { } // false  
const a = { }; const b = a;  
a === b    // true
```

Q45. What are Symbols in JavaScript, and when are they useful?

Symbol is a primitive data type that creates unique values. Symbols are commonly used as object property keys when you want to avoid accidental name collisions or define well-known protocol properties such as Symbol.iterator.

Example

```
const id = Symbol("id");  
const user = { [id]: 123 };
```

Q46. What is the purpose of the “arguments” object in JavaScript?

The arguments object is an array-like object available inside regular functions that contains the arguments passed to the function. It is not available in arrow functions. In modern code, rest parameters (...args) are usually preferred because they create a real array.

Example

```
function sum() {  
  return [...arguments].reduce((a, b) => a + b, 0);  
}
```

Q47. What are the different ways to define methods in JavaScript objects?

Methods can be defined with method shorthand, function expressions, or arrow functions assigned as properties. In classes, methods are also defined directly in the class body. Method syntax is usually preferred for object methods when you need a normal dynamic this.

Example

```
const user = {  
  greet() { console.log("Hi"); },  
  bye: function() { console.log("Bye"); }  
};
```

Q48. Explain the concept of memoization and its benefits.

Memoization is an optimization technique that caches the result of a function call so repeated calls with the same inputs can return the stored result instead of recalculating it. It can improve performance for expensive, repeated computations at the cost of additional memory.

Example

```
const cache = new Map();  
// store and reuse results by input
```

Q49. What is the difference between “slice” and “splice” in JavaScript arrays?

slice() returns a portion of an array without changing the original array. splice() changes the original array by adding, removing, or replacing elements. slice is commonly used for non-mutating copies; splice is for in-place changes.

Example

```
const a = [1, 2, 3, 4];  
a.slice(1, 3); // [2, 3]  
a.splice(1, 2); // removes 2, 3
```

Q50. What is the purpose of the “apply” and “call” methods in JavaScript?

call() and apply() invoke a function immediately with a specified this value. The main difference is that call() receives arguments individually, while apply() receives them as an array-like collection. Both are useful for controlling function context.

Example

```
fn.call(obj, 1, 2);  
fn.apply(obj, [1, 2]);
```

06. Modern Syntax, JSON & Common Array Techniques

Questions 51-60

Q51. Explain the concept of the event loop in JavaScript and how it handles asynchronous operations.

JavaScript runs synchronous code on the call stack while asynchronous work such as timers, network operations, and events is handled outside the stack. When the stack is empty, the event loop moves queued work into the stack. Promise callbacks use the microtask queue and are processed before the next task such as a timer callback.

Q52. What are arrow functions, and how are they different from regular functions?

Arrow functions provide shorter function syntax and use lexical this, meaning they inherit this from the surrounding scope. Unlike regular functions, they do not have their own arguments object and cannot be used as constructors with new.

Example

```
const add = (a, b) => a + b;
```

Q53. What is the purpose of the “filter” method in JavaScript?

filter() creates a new array containing only the elements that satisfy a condition. It is used when you want to select items without modifying the original array.

Example

```
const even = [1, 2, 3, 4].filter(n => n % 2 === 0);  
// [2, 4]
```

Q54. Explain the concept of destructuring in JavaScript.

Destructuring is syntax for extracting values from arrays or properties from objects into variables. It makes code more concise and is commonly used with function parameters, API responses, and imports.

Example

```
const user = { name: "John", age: 25 };  
const { name, age } = user;
```

Q55. What is callback hell, and how can it be avoided?

Callback hell is deeply nested callback-based asynchronous code that becomes difficult to read, maintain, and debug. It can be reduced by using named functions, modularizing work, and, in modern JavaScript, using Promises and `async/await`.

Example

```
async function run() {  
  const data = await loadData();  
  return processData(data);  
}
```

Q56. What are promises, and how do they work in JavaScript?

A Promise represents the eventual completion or failure of an asynchronous operation. It starts in pending, then becomes fulfilled or rejected. You handle the result with `then()`, errors with `catch()`, and cleanup with `finally()`.

Q57. Explain the difference between synchronous and asynchronous JavaScript.

Synchronous code runs sequentially and blocks the current execution flow until each operation finishes. Asynchronous code allows longer operations to complete later so other work can continue, typically using callbacks, Promises, or `async/await`.

Q58. What is JSON, and how is it used in JavaScript?

JSON stands for JavaScript Object Notation. It is a text format commonly used to exchange structured data between applications. JavaScript converts JSON text to objects with `JSON.parse()` and objects to JSON text with `JSON.stringify()`.

Example

```
const obj = JSON.parse('{"name":"John"}');  
const text = JSON.stringify(obj);
```

Q59. How can you remove duplicates from an array in JavaScript?

A common approach is to convert the array to a Set and then back to an array. For objects, duplicates require a key-based or custom comparison because Set only removes duplicate references.

Example

```
const unique = [...new Set([1, 2, 2, 3])];  
// [1, 2, 3]
```

Q60. What is the purpose of the “includes” method in JavaScript?

includes() checks whether an array or string contains a specified value or substring. It returns true or false and is useful for straightforward membership checks.

Example

```
[1, 2, 3].includes(2); // true  
"hello".includes("ell"); // true
```

07. Modules, Web APIs, Objects & Arrays**Questions 61-70**

Q61. How do you handle default values in JavaScript functions?

Default parameters let you provide a fallback value when an argument is undefined. They make function inputs safer and reduce the need for manual checks.

Example

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}
```

Q62. What are template literals, and how are they used?

Template literals are strings written with backticks. They support interpolation with \${} and can span multiple lines, making them useful for dynamic strings and formatted text.

Example

```
const name = "John";  
const msg = `Hello, ${name}!`;
```

Q63. Explain the concept of modules in JavaScript.

Modules let you split code into separate files with their own scope and explicit dependencies. JavaScript modules use export to expose values and import to consume them. Modules improve organization, reuse, and maintainability.

Example

```
export function add(a, b) { return a + b; }  
import { add } from "./math.js";
```

Q64. What is CORS, and why is it important?

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page can make requests to a different origin. Servers use response headers to allow specific cross-origin requests.

Example

```
Access-Control-Allow-Origin: https://example.com
```

Q65. What is the “fetch” API, and how does it work?

The Fetch API provides a Promise-based way to make HTTP requests. `fetch()` returns a Promise for a Response object, which you can inspect for status information and parse with methods such as `json()`. HTTP error status codes do not automatically reject the Promise, so they should be checked explicitly.

Example

```
const res = await fetch("/api/users");
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

Q66. Explain the concept of immutability in JavaScript.

Immutability means treating existing data as unchanged and creating new values when updates are needed. It helps make state changes predictable and reduces unintended side effects, especially in application state management.

Example

```
const updated = { ...user, name: "Jane" };
```

Q67. What is the purpose of the “find” method in JavaScript arrays?

`find()` returns the first array element that satisfies a condition. If no element matches, it returns undefined. It is useful when you need the actual matching item rather than an array of matches.

Example

```
const user = users.find(u => u.id === 10);
```

Q68. How do “import” and “export” work in JavaScript?

`export` makes variables, functions, classes, or other values available from a module. `import` brings those exported values into another module. JavaScript supports named exports and a default export.

Example

```
export const apiUrl = "/api";
import { apiUrl } from "./config.js";
```

Q69. What is a Proxy object in JavaScript?

A Proxy wraps another object and lets you intercept operations such as reading, writing, deleting, or checking properties. It is useful for validation, logging, reactive systems, and customizing object behavior.

Example

```
const proxy = new Proxy(user, {
  get: (target, key) => target[key]
});
```

Q70. How do you flatten an array in JavaScript?

You can flatten arrays with `flat()`, which removes nested levels up to a specified depth. `flat(Infinity)` can flatten all nested levels, though using the required depth is usually clearer.

Example

```
const arr = [1, [2, [3]]];  
arr.flat(2); // [1, 2, 3]
```

08. Browser APIs, Collections & Object Utilities

Questions 71-80

Q71. What are IIFE (Immediately Invoked Function Expressions)?

An IIFE is a function expression that is defined and executed immediately. It was commonly used to create a private scope and avoid polluting the global namespace before JavaScript modules became standard.

Example

```
(function () {  
  const secret = 42;  
})();
```

Q72. What is the purpose of the “setTimeout” and “setInterval” functions?

`setTimeout()` schedules a function to run once after at least the specified delay. `setInterval()` schedules repeated execution until it is cleared. Both are asynchronous timers and do not block the main thread while waiting.

Example

```
const id = setInterval(() => console.log("tick"), 1000);  
clearInterval(id);
```

Q73. Explain the concept of polyfills in JavaScript.

A polyfill is code that provides support for a modern JavaScript feature in environments that do not implement it natively. Polyfills often add a method or API to older environments while keeping the application code consistent.

Example

```
if (!Array.prototype.includes) { /* polyfill implementation */ }
```

Q74. What is a WeakMap in JavaScript, and how is it different from a Map?

`WeakMap` stores key-value pairs where keys must be objects or non-registered symbols, and the keys are held weakly so they can be garbage-collected when nothing else references them. Unlike `Map`, `WeakMap` is not directly iterable and does not expose `size`.

Q75. How do you handle data binding in JavaScript?

Data binding means keeping JavaScript data and the user interface synchronized. In plain JavaScript, this is usually done by reading input values, updating state, and rendering changes to the DOM. Frameworks such as React and Angular provide higher-level patterns for this.

Example

```
input.addEventListener("input", e => {  
  state.name = e.target.value;  
  render();  
});
```

Q76. What is the purpose of the “every” and “some” methods in JavaScript arrays?

`every()` returns true only if all elements pass the test. `some()` returns true if at least one element passes. Both are useful for concise condition checks and stop early when the result is known.

Example

```
[2, 4, 6].every(n => n % 2 === 0); // true  
[1, 3, 4].some(n => n % 2 === 0); // true
```

Q77. What is `Object.create()` and when would you use it?

`Object.create()` creates a new object with a specified prototype. It is useful when you want explicit prototype-based inheritance or when creating objects without inheriting from `Object.prototype`.

Example

```
const parent = { greet() {} };  
const child = Object.create(parent);
```

Q78. How do you remove properties from an object in JavaScript?

The delete operator removes a property from an object. For immutable code, you can create a new object that excludes the property using destructuring and rest syntax.

Example

```
delete user.password;  
  
const { password, ...safeUser } = user;
```

Q79. What is the purpose of the “`Object.freeze`” method?

`Object.freeze()` prevents an object from being changed directly: existing properties cannot be added, removed, or reassigned. It is shallow, so nested objects are not automatically frozen.

Example

```
const config = Object.freeze({ mode: "prod" });  
// config.mode = "dev"; // ignored or throws in strict mode
```

Q80. How do you check if an object is empty in JavaScript?

A common approach is `Object.keys(obj).length === 0`. This checks whether the object has no own enumerable string-keyed properties.

Example

```
const isEmpty = Object.keys(obj).length === 0;
```

09. Performance, Inheritance & Advanced Functions

Questions 81-90

Q81. Explain the concept of throttling in JavaScript.

Throttling limits how often a function can execute within a given time interval. It is useful for high-frequency events such as scroll, resize, and mouse movement when you want regular but controlled execution.

Example

```
window.addEventListener("scroll", throttle(handleScroll, 200));
```

Q82. What is a WeakSet in JavaScript?

WeakSet stores only objects or non-registered symbols as weakly held members. Members can be garbage-collected when they are no longer referenced elsewhere. Like WeakMap, WeakSet is not iterable and does not expose size.

Example

```
const seen = new WeakSet();  
seen.add(object);
```

Q83. How does JavaScript's garbage collection work?

JavaScript engines automatically reclaim memory for objects that are no longer reachable from active program roots such as global variables, the stack, or other live objects. Modern engines commonly use tracing techniques such as mark-and-sweep with generational optimizations.

Q84. What is debouncing in JavaScript, and how is it different from throttling?

Debouncing delays execution until a period of inactivity has passed, so repeated calls reset the timer. Throttling allows execution at a controlled maximum frequency. Debouncing is common for search input; throttling is common for scroll or resize handlers.

Example

```
input.addEventListener("input", debounce(search, 300));
```

Q85. What is the purpose of the “Object.assign” method?

Object.assign() copies enumerable own properties from one or more source objects into a target object. It performs a shallow copy and mutates the target object.

Example

```
const merged = Object.assign({}, defaults, config);
```

Q86. Explain the difference between map, filter, and reduce.

map() transforms every element and returns a new array of the same length. filter() selects elements that satisfy a condition and may return a shorter array. reduce() combines elements into a single accumulated result, which can be any type.

Example

```
nums.map(f);  
nums.filter(test);  
nums.reduce(combine, initial);
```

Q87. How do you handle inheritance in JavaScript?

JavaScript supports inheritance primarily through the prototype chain. Modern code can use class and extends syntax as a cleaner abstraction over prototypal inheritance. Object.create() provides direct prototype-based inheritance.

Example

```
class Admin extends User {  
  deleteUser() {}  
}
```

Q88. What is the difference between synchronous and asynchronous functions?

A synchronous function completes its work before control moves to the next statement. An asynchronous function can start work that completes later and, with async, returns a Promise. The key difference is how completion is scheduled and how the caller receives the result.

Q89. How does the “async” keyword work with functions?

Declaring a function with async makes it return a Promise. Inside that function, await can pause execution until a Promise settles, while the rest of the program can continue running. Rejected Promises can be handled with try...catch or by the caller.

Example

```
async function getUser() {  
  const res = await fetch("/api/user");  
  return res.json();  
}
```

Q90. What are JavaScript decorators?

Decorators are a syntax for applying behavior or metadata to classes and class elements such as methods or fields. They are useful for cross-cutting concerns such as logging, validation, or dependency injection. Modern JavaScript standardizes decorators, while support depends on the runtime and toolchain being used.

10. Production Concepts & Promise Patterns

Questions 91-100

Q91. Explain how to optimize JavaScript code for performance.

Common techniques include reducing unnecessary DOM work, avoiding repeated expensive calculations, minimizing allocations, using efficient data structures, splitting or lazy-loading code, debouncing or throttling high-frequency events, and measuring bottlenecks with profiling tools before optimizing.

Q92. What is a service worker in JavaScript?

A service worker is a background script that runs separately from the web page and can intercept network requests, cache resources, enable offline behavior, and support features such as push notifications. It operates within security and scope restrictions.

Example

```
navigator.serviceWorker.register("/sw.js");
```

Q93. How do you use “try...catch” for error handling in JavaScript?

try...catch lets you handle exceptions without crashing the current operation. Put code that may throw inside try, handle the error inside catch, and use finally for cleanup that must run whether the operation succeeds or fails.

Example

```
try {
  const data = JSON.parse(text);
} catch (error) {
  console.error("Invalid JSON", error);
} finally {
  cleanup();
}
```

Q94. Explain how promises can be chained.

Promise chaining works by returning a value or another Promise from a then() callback. The returned result becomes the input to the next then(). A catch() can handle failures from earlier steps, and finally() can perform cleanup.

Example

```
fetch("/api/user")
  .then(r => r.json())
```

```
.then(user => loadPosts(user.id))  
.catch(console.error);
```

Q95. What are the benefits of using closures in JavaScript?

Closures allow functions to preserve access to surrounding variables. They are useful for data privacy, maintaining state between calls, creating function factories, callbacks, and implementing patterns such as memoization.

Example

```
function createCounter() {  
  let count = 0;  
  return () => ++count;  
}
```

Q96. How can you reverse a string in JavaScript?

For simple strings, you can split the string into an array of code units, reverse it, and join it again. For Unicode text containing surrogate pairs or grapheme clusters, [...str] or Array.from(str) is safer than split().

Example

```
const reversed = [...str].reverse().join("");
```

Q97. What is a Set in JavaScript?

Set is a collection of unique values. It stores each value only once and provides methods such as add(), has(), and delete(). It is commonly used for membership checks and removing duplicates from arrays.

Example

```
const unique = new Set([1, 2, 2, 3]);  
// Set {1, 2, 3}
```

Q98. Explain the concept of promises with async/await.

async/await is syntax built on top of Promises. An async function returns a Promise, and await waits for a Promise to settle inside that function. This makes asynchronous code read more like synchronous code while retaining non-blocking behavior.

Example

```
async function load() {  
  try {  
    const data = await fetchData();  
    return data;  
  } catch (error) {}  
}
```

Q99. How do you use “Object.keys,” “Object.values,” and “Object.entries” in JavaScript?

Object.keys() returns an array of an object's own enumerable string keys. Object.values() returns the corresponding values. Object.entries() returns [key, value] pairs and is useful for iterating over both keys and values.

Example

```
Object.keys(user);  
Object.values(user);  
Object.entries(user);
```

Q100. What is the purpose of “Promise.all” in JavaScript?

Promise.all() runs multiple Promises concurrently and fulfills with an array of results when all of them fulfill. It rejects as soon as any input Promise rejects. It is useful when several independent asynchronous operations are all required.

Example

```
const [users, posts] = await Promise.all([  
  fetchUsers(),  
  fetchPosts()  
]);
```

JavaScript Interview Preparation

Revise the answer. Say it aloud. Then expand only when the interviewer asks for more depth.